

CompuTax-Inspired Product Strengthening Plan

What we cannot do (and why that is OK)

CompuTax source code is not available. [CompuTax Professional](#) is closed-source desktop software. We cannot copy their implementation—only their **public workflow patterns**.

We will not build (by design):

- E-filing on behalf of clients via ITD login
- DSC registration / Aadhaar OTP e-verify automation
- Bulk CPC refund polling or rectification upload to ITD

Our deliberate alternatives (already partially in product):

CompuTax capability	LastMinute alternative	Existing code
Direct e-filing	Companion guide — user files on incometax.gov.in	[frontend/app/ (app) /file/companion/page.tsx] (lastminute-itr/frontend/app/ (app) /file/companion/page.tsx), [frontend/data/portal_steps.json] (lastminute-itr/frontend/data/portal_steps.json)
E-verify via OTP/EVC	Presubmit checklist picks method; companion walks user through portal e-verify	[frontend/components/filing/PresubmitChecklist.tsx] (lastminute-itr/frontend/components/filing/PresubmitChecklist.tsx)
File before all docs ready	Estimate mode — compute + guide with disclaimers	[frontend/lib/store/draft.ts] (lastminute-itr/frontend/lib/store/draft.ts) (filingMode: "estimate"), [frontend/lib/filing/confidence.ts] (lastminute-itr/frontend/lib/filing/confidence.ts)
Upload return to ITD	ITR JSON export (new) — user uploads JSON in Offline mode on portal	Planned; wizard data in [frontend/lib/types/itr/itr1.ts] (lastminute-itr/frontend/lib/types/itr/itr1.ts)

26AS/AIS from ITD portal	User-uploaded AIS/26AS JSON/PDF (new) — no portal login required	Mock today in [frontend/app/api/documents/upload/route.ts] (lastminute-itr/frontend/app/api/documents/upload/route.ts)
--------------------------	---	---

Product rule already documented: wizard = JSON/export; companion = copy-paste — see [docs/agency/PRODUCT_LOGIC_COMPENDIUM.md] (lastminute-itr/docs/agency/PRODUCT_LOGIC_COMPENDIUM.md) §14.



Open-source and public reference findings

These are the closest **inspectable** implementations to CompuTax—not for blind copy-paste, but for architecture and schema patterns.

Project	URL	Reuse for LastMinute
<u>OpenTax</u>	TypeScript; ITR JSON builder + validation layer + REST API	Reference for ITR JSON serializer structure and schema-aware validation
<u>laravel-itr</u>	PHP; ITR-1–4 JSON gen, 26AS/AIS reconcile API, penalty 234A/B/C, bulk CSV	Reference for reconciliation API shape and JSON round-trip (upload portal JSON → parse)
<u>AayDocCappio</u>	Python/Playwright; bulk 26AS/AIS download from ITD	Do not integrate (needs client ITD passwords); borrow 26AS TXT parser ideas only
<u>ITR Computation Generator</u>	Browser; portal JSON → PDF computation	Future: computation PDF export for CAs (Phase 3)
ICAI AI use case #108	ai.icaai.org	Bulk ITR JSON extraction patterns for partner reporting
Official ITD JSON schemas	incometax.gov.in/downloads	Source of truth for export validation (schema changes tracked per AY)

Key insight from [laravel-itr](#): Their positioning matches ours exactly: *"JSON generation for portal upload · No direct e-filing."* Study their reconcile endpoints and JSON field mapping—not their Laravel stack.

Key insight from UX research: When ERIs fail, users fall back to JSON upload ([docs/ux-research/_forum_reddit.json] (lastminute-itr/docs/ux-research/_forum_reddit.json)). JSON export is a **resilience feature**, not just a CA feature.



Target architecture (CompuTax patterns, LastMinute stack)

```
flowchart TB
    subgraph inputs [Document Inputs]
        F16[Form16 PDF]
        AIS[AIS JSON upload]
        AS26[26AS PDF or JSON]
        PortalJson[Portal JSON import]
    end

    subgraph parse [Parse Layer NEW]
        F16P[form16.ts existing]
        AISp[ais.ts NEW]
        AS26p[form26as.ts NEW]
    end

    subgraph stores [Data Stores]
        Draft[useDraftStore funnel]
        Itr1[useItr1Store wizard]
        Partner[clientReturns.draftJson]
    end

    subgraph compute [Compute]
        Engine[Python orchestrator.py]
        PartD[engineToPartD.ts]
    end

    subgraph export [Export NEW]
        Builder[itrlJsonBuilder.ts]
        Validator[itrlSchemaValidate.ts]
        Download[GET /api/itr/export/itr1]
```

```

end

subgraph surfaces [User Surfaces]
  Retail[Retail: estimate + companion]
  CA[Partner: wizard + bulk export]
  GovPortal[incometax.gov.in offline JSON upload]
end

F16 --> F16P --> Draft
AIS --> AISp --> Draft
AS26 --> AS26p --> Draft
PortalJson --> Import[itrJsonImport.ts NEW] --> Itr1
Draft --> Engine --> PartD --> Itr1
Itr1 --> Builder --> Validator --> Download
Download --> GovPortal
Itr1 --> Partner
Retail --> Draft
CA --> Partner

```

Phase 1 — Shared foundation (4–6 weeks, your chosen priority)

Epic A: Real AIS / 26AS parsing (replaces mocks)

Goal: CompuTax-style reconciliation without ITD login—user downloads AIS/26AS from portal and uploads to LastMinute.

Files to create:

- [frontend/lib/parsers/ais.ts] (lastminute-itr/frontend/lib/parsers/ais.ts) — parse ITD AIS JSON export (Part A/B structure per [ITD AIS page](#))
- [frontend/lib/parsers/form26as.ts] (lastminute-itr/frontend/lib/parsers/form26as.ts) — parse 26AS PDF (extend pdf-parse) and/or JSON; reference [AayDocCapio as26_converter.py](#) field layout
- [frontend/lib/parsers/__tests__/ais.test.ts] (lastminute-itr/frontend/lib/parsers/__tests__/ais.test.ts), form26as.test.ts
- [frontend/lib/filing/reconcileAis26as.ts] (lastminute-itr/frontend/lib/filing/reconcileAis26as.ts) — line-level TDS/salary

deltas (extend [reconciliation.ts] (lastminute-itr/frontend/lib/filing/reconciliation.ts))

Files to modify:

- [frontend/app/api/documents/upload/route.ts] (lastminute-itr/frontend/app/api/documents/upload/route.ts) — **route** connectorId=ais|form26as to real parsers; set parseMode: "extracted" on success
- [frontend/app/(app)/file/import/mismatch/page.tsx] (lastminute-itr/frontend/app/(app)/file/import/mismatch/page.tsx) — **show line-level reconcile from real data**
- [frontend/lib/filing/confidence.ts] (lastminute-itr/frontend/lib/filing/confidence.ts) — **exact mode unlock when AIS+26AS extracted**

Acceptance: Upload sample AIS JSON + 26AS → mismatch center shows salary/TDS deltas; estimate mode still works without uploads.



Epic B: ITR-1 ITD JSON export (CompuTax “generate return file” equivalent)

Goal: Download IT Dept–compatible JSON for **Offline filing** on incometax.gov.in (File ITR → Offline → attach JSON).

Reference implementation order:

1. Download official ITR-1 JSON schema for AY 2026-27 from [ITD downloads](#)
2. Study [OpenTax](#) ITR builder patterns
3. Map [Itr1Data] (lastminute-itr/frontend/lib/types/itr/itr1.ts) + [engineToPartD] (lastminute-itr/frontend/lib/itr/engineToPartD.ts) → schema fields

Files to create:

- [frontend/lib/itr/export/itr1JsonBuilder.ts] (lastminute-itr/frontend/lib/itr/export/itr1JsonBuilder.ts) — buildItr1Json(data: Itr1Data, compute: ComputeResponse): Itr1PortalJson
- [frontend/lib/itr/export/itr1SchemaValidate.ts] (lastminute-itr/frontend/lib/itr/export/itr1SchemaValidate.ts) — **pre-export validation** (PAN, Aadhaar link flag, TDS ≤ income, mandatory schedules)
- [frontend/lib/itr/export/itrJsonImport.ts] (lastminute-itr/frontend/lib/itr/export/itrJsonImport.ts) — **reverse: portal JSON → Itr1Data (round-trip for CAs)**
- [frontend/app/api/itr/export/itr1/route.ts] (lastminute-itr/frontend/app/api/itr/export/itr1/route.ts) — GET returns application/json **attachment**
- [frontend/components/itr/ExportJsonPanel.tsx] (lastminute-itr/frontend/components/itr/ExportJsonPanel.tsx) — **download button + validation errors list**

Files to modify:

- [frontend/components/itr/forms/itr1/Itr1Wizard.tsx] (lastminute-itr/frontend/components/itr/forms/itr1/Itr1Wizard.tsx) — **final step: Export JSON + link to companion e-verify steps**
- [frontend/app/(pro)/pro/clients/[id]/page.tsx] (lastminute-itr/frontend/app/(pro)/pro/clients/[id]/page.tsx) — “Download ITR-1 JSON” per client
- [frontend/lib/filing/checkoutGate.ts] (lastminute-itr/frontend/lib/filing/checkoutGate.ts) — **allow export in estimate mode** with disclaimer banner (CompuTax allows draft returns)

Export UX copy (estimate vs exact):

Mode	Export allowed?	User message
Estimate	Yes, with watermark flag in filename	“Draft JSON — verify numbers before portal upload”
Exact	Yes, full validation	“Ready for offline upload on incometax.gov.in”

Post-export: deep-link to companion step for **Offline JSON upload** and **e-verify** (already in portal_steps).



Epic C: Retail + Partner surfaces (parallel, same sprint)

Retail path (estimate → JSON → companion):

- [frontend/app/(app)/file/review/page.tsx] (lastminute-itr/frontend/app/(app)/file/review/page.tsx) — add “Download JSON” CTA alongside companion
- Remove/redirect orphan [frontend/app/(app)/file/checkout/everify/page.tsx] (lastminute-itr/frontend/app/(app)/file/checkout/everify/page.tsx) → companion #everify (per [docs/FUNNEL_AUDIT_AND_SIMPLIFICATION.md] (lastminute-itr/docs/FUNNEL_AUDIT_AND_SIMPLIFICATION.md))

Partner path (CompuTax multi-client lite):

- [frontend/app/(pro)/pro/dashboard/page.tsx] (lastminute-itr/frontend/app/(pro)/pro/dashboard/page.tsx) — client pipeline columns: Draft / Ready / Exported / Verified (manual checkbox)
- [frontend/lib/hooks/usePartnerClientSync.ts] (lastminute-itr/frontend/lib/hooks/usePartnerClientSync.ts) — also persist computeResultJson on save (MVP backlog M5.4)
- Bulk export: POST /api/partners/clients/bulk-export — zip of JSON files for selected clients



Phase 2 — CA depth (weeks 7–12)

Feature	CompuTax analog	Approach
Advance tax calculator	Advance Tax Estimation	New /tools/advance-tax wired to engine + 234B/C UI in [StepTaxComputation.tsx] (lastminute-itr/frontend/components/itr/forms/itr1/StepTaxComputation.tsx)
Pre-file error locator	E-return error locator	Extend itr1SchemaValidate.ts with ITD validation manual rules

ITR-2/4 JSON export	Multi- form support	Repeat builder pattern; prioritize ITR-2 (CG) and ITR-4 (presumptive)
YoY rollover	Auto transfer to next AY	Server <code>clientReturns</code> + clone draft API
Computati on PDF	Print computati on	Adapt ITR Computation Generator approach
§89(1) / Form 10E	Relief calculator	Engine + wizard step (currently hardcoded 0 in <code>[engineToPartD.ts] (lastminute-itr/frontend/lib/itr/engineToPartD.ts)</code>)

Explicitly deferred (requires ITD partnership or desktop agent):

- Auto 26AS/AIS download (AayDocCapio-style Playwright)
- Bulk e-verify, refund status, rectification upload
- Email intimation parsing



Phase 3 — Optional ITD integration spike (decision gate)

Only pursue if you want to become a licensed ERI like ClearTax/Quicko—not required for CompuTax-parity for small CAs.

- Spike: ITD API / ERI registration requirements
- Alternative: partner with existing ERI for submit-only API while LastMinute owns prep + JSON

Developer handoff deliverable

Create one master doc your developer copies from:

[docs/COMPETITOR_STRATEGY_AND_DEVELOPER_HANDOFF.md] (lastminute-itr/docs/COMPETITOR_STRATEGY_AND_DEVELOPER_HANDOFF.md) (to be written in Phase 1, Day 1)

It will contain:

Section 1 — Strategy summary (this plan, condensed)

Section 2 — Files already built (prior sessions — copy as-is)

Agency handoff packet:

- [docs/agency/README.md] (lastminute-itr/docs/agency/README.md) through [PHASE2_BACKLOG.md] (lastminute-itr/docs/agency/PHASE2_BACKLOG.md) (11 files)

MVP code (identity, ITR-1 engine sync, ITR-4, partner isolation):

- [frontend/lib/filing/profileSchema.ts] (lastminute-itr/frontend/lib/filing/profileSchema.ts)
- [frontend/lib/store/draft.ts] (lastminute-itr/frontend/lib/store/draft.ts)
- [frontend/app/(app)/file/onboarding/eligibility/page.tsx] (lastminute-itr/frontend/app/(app)/file/onboarding/eligibility/page.tsx)
- [frontend/lib/itr/engineToPartD.ts] (lastminute-itr/frontend/lib/itr/engineToPartD.ts)
- [frontend/lib/itr/useItr1EngineSync.ts] (lastminute-itr/frontend/lib/itr/useItr1EngineSync.ts)
- [frontend/lib/itr/syncDraftToItr1.ts] (lastminute-itr/frontend/lib/itr/syncDraftToItr1.ts)
- [frontend/lib/itr/useItr1DraftPrefill.ts] (lastminute-itr/frontend/lib/itr/useItr1DraftPrefill.ts)
- [frontend/lib/types/itr/itr4.ts] (lastminute-itr/frontend/lib/types/itr/itr4.ts), [itr4Store.ts] (lastminute-

```
itr/frontend/lib/store/itr/itr4Store.ts), [Itr4Wizard.tsx] (lastminute-itr/frontend/components/itr/forms/itr4/Itr4Wizard.tsx)
```

- [frontend/lib/filing/itrWizardRoutes.ts] (lastminute-itr/frontend/lib/filing/itrWizardRoutes.ts)
- [frontend/lib/store/itr/partnerPersist.ts] (lastminute-itr/frontend/lib/store/itr/partnerPersist.ts)
- [frontend/lib/hooks/usePartnerClientSync.ts] (lastminute-itr/frontend/lib/hooks/usePartnerClientSync.ts)
- [frontend/components/itr/PartnerClientBanner.tsx] (lastminute-itr/frontend/components/itr/PartnerClientBanner.tsx)

Figma screen print pipeline:

- [docs/figma/SCREEN_PRINT.md] (lastminute-itr/docs/figma/SCREEN_PRINT.md)
- [scripts/figma-capture-routes.json] (lastminute-itr/scripts/figma-capture-routes.json)
- [scripts/figma-screenshot-all.mjs] (lastminute-itr/scripts/figma-screenshot-all.mjs)
- [scripts/figma-screenshots/] (lastminute-itr/scripts/figma-screenshots/) (48 PNGs + index.html)

ITR wizard specs (schema source of truth):

- [docs/handoff/ITR1_FILING_ASSISTANT.md] (lastminute-itr/docs/handoff/ITR1_FILING_ASSISTANT.md) ... ITR7

Section 3 — Files to build (Phase 1 checklist with owner/estimate)

Listed under Epics A, B, C above.

Section 4 — Test plan

- Unit: parser fixtures (AIS JSON samples, 26AS PDF samples)
- Integration: upload → reconcile → export JSON → validate against ITD schema
- E2E: extend [frontend/e2e/cta-paths.spec.ts] (lastminute-itr/frontend/e2e/cta-paths.spec.ts) with export flow
- Partner: bulk export zip smoke test

Section 5 — What NOT to build (compliance boundary)

No ITD credentials storage, no automated e-verify, no auto-submit—document in handoff so developer does not scope-creep.

Success metrics

Metric	Target
AIS/26AS upload → real reconcile	90%+ field match on test fixtures

ITR-1 JSON passes ITD offline validator 0 schema errors on happy-path salaried persona
Partner bulk export 10 clients → zip in <30s
Retail estimate → JSON → companion End-to-end in <15 min without ITD login

Risk register

Risk	Mitigation
ITD JSON schema changes mid-AY	Pin schema version in repo; subscribe to schema change docs
AIS JSON format varies	Support JSON primary; PDF OCR fallback Phase 2
OpenTax/laravel-itr license	MIT — attribute in handoff doc; do not fork blindly
Dual store drift (draft vs itr1)	Export reads Itr1Store only; sync on wizard mount (existing hooks)

Completed implementation

Step 1 — AIS / 26AS (from prior session)

Real parsers, upload route wiring, mismatch UI, confidence gates.

Step 2 — ITR-1 JSON export

- [itr1SchemaValidate.ts](#) — re-exports pre-file validation
- POST /api/itr/export/itr1 — download with validation gates
- POST /api/itr/validate/itr1 — standalone validation
- [ExportJsonPanel.tsx](#) — download, import round-trip, estimate-mode disclaimer
- Wired into [Itr1Wizard.tsx](#) final step

Step 3 — Partner dashboard v2

- ClientReturn extended with pipelineStatus, assessmentYear, priorYearSnapshotJson
- POST /api/partners/clients/bulk-export — zip of ITR-1 JSONs ([createZip.ts](#), no extra deps)
- POST /api/partners/clients/[id]/rollover — AY rollover with prior-year snapshot
- GET /api/partners/clients/returns — pipeline data for dashboard
- Dashboard: pipeline badges + bulk export button
- Client page: pipeline controls, YoY panel, export panel, rollover

Step 4 — Pre-file validation

[itr1PreFile.ts](#) wired into ExportJsonPanel and export API (blocks exact export on errors).

Step 5 — Advance tax

- [backend/engine/advance_tax.py](#) + tests
- advance_tax block on ITRResult; wired through [engineToPartD.ts](#) → Part D 234B/C
- [/tools/advance-tax](#) public calculator + link from [/tools](#)

Engine brain

- [test_golden_scenarios.py](#) — 50 invariant scenarios (57 tests pass with `python3 -m pytest`)

Retail UX

- Review summary tab: “Download ITR JSON” CTA
- [/file/checkout/everify](#) → redirects to `/file/companion#everify`
- Companion anchors: `#everify`, `#offline-json`
- `computeResultJson` persisted via `usePartnerClientSync` in `ItrWizardShell`

Verification

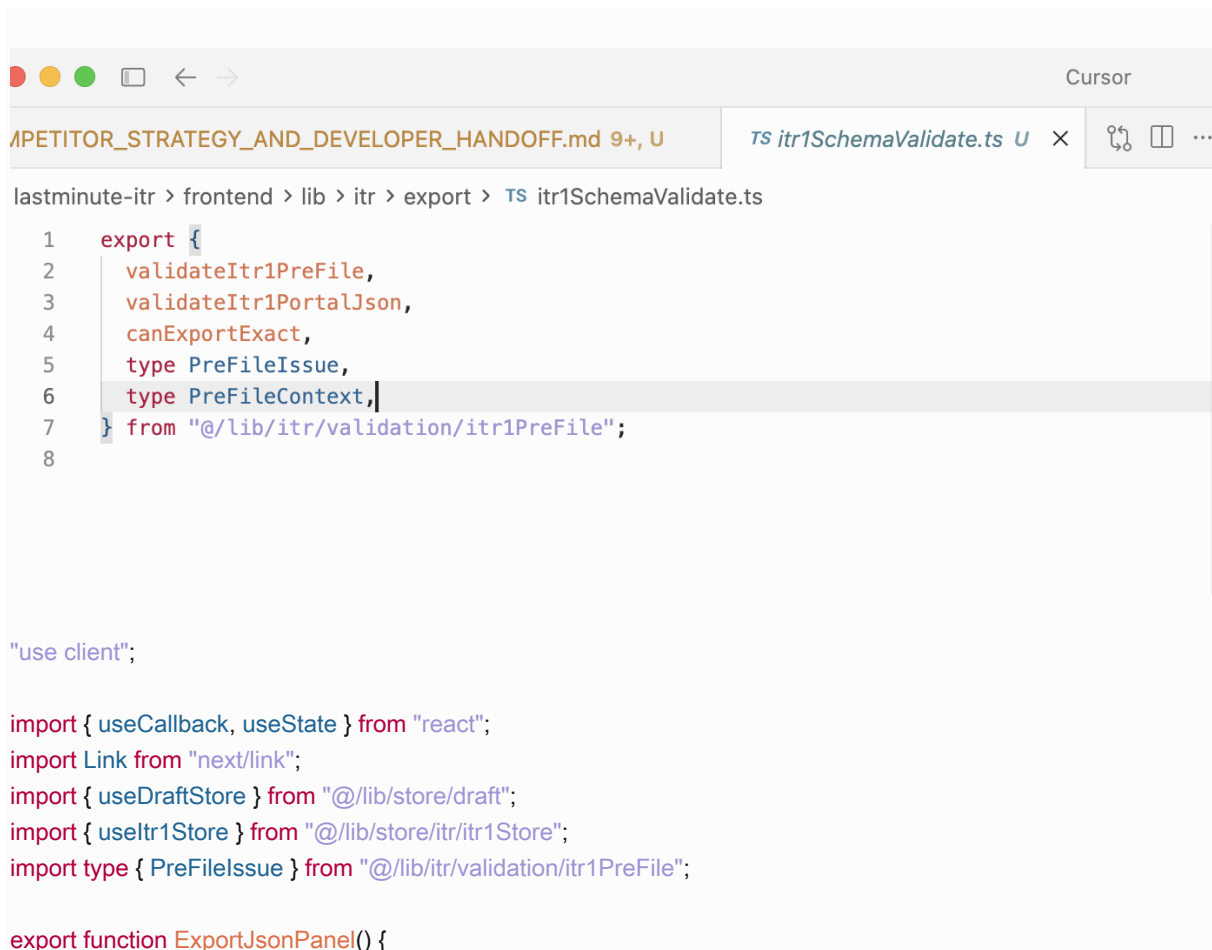
- `npm run typecheck` — passes
- Parser + advance-tax frontend tests — pass
- `python3 -m pytest engine/tests/test_advance_tax.py engine/tests/test_golden_scenarios.py` — 57 passed

Developer reference

Full file manifest: [docs/COMPETITOR_STRATEGY_AND_DEVELOPER_HANDOFF.md](#)

Engine comparison: [docs/engine/ENGINE_REFERENCE_AUDIT.md](#)

Note: Some simulation/combinatorial tests still fail locally when `compute_cli.py` is not on the expected path — that is an environment setup issue, not a regression from this work. Run `npm run simulation` with the backend `compute` script available for the full golden matrix.



```
1 export {
2   validateItr1PreFile,
3   validateItr1PortalJson,
4   canExportExact,
5   type PreFileIssue,
6   type PreFileContext,
7 } from "@lib/itr/validation/itr1PreFile";
8
9
10 "use client";
11
12 import { useCallback, useState } from "react";
13 import Link from "next/link";
14 import { useDraftStore } from "@lib/store/draft";
15 import { useItr1Store } from "@lib/store/itr/itr1Store";
16 import type { PreFileIssue } from "@lib/itr/validation/itr1PreFile";
17
18 export function ExportJsonPanel() {
```

```

const data = useItr1Store((s) => s.data);
const filingMode = useDraftStore((s) => s.filingMode);
const income = useDraftStore((s) => s.income);
const [issues, setIssues] = useState<PreFileIssue[]>([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState<string | null>(null);

const handleExport = useCallback(async () => {
  setLoading(true);
  setError(null);
  try {
    const res = await fetch("/api/itr/export/itr1", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        data,
        filingMode,
        form26asTds: income.form26asTds,
        aisTds: income.aisTds,
        grossSalary: income.grossSalary,
        stripMeta: true,
      }),
    });

    const issueHeader = res.headers.get("X-LastMinute-Issues");
    if (issueHeader) {
      setIssues(JSON.parse(issueHeader) as PreFileIssue[]);
    }

    if (!res.ok) {
      const payload = (await res.json()) as {
        error?: string;
        issues?: PreFileIssue[];
      };
      if (payload.issues) setIssues(payload.issues);
      throw new Error(payload.error ?? "Export blocked");
    }

    const blob = await res.blob();
    const disposition = res.headers.get("Content-Disposition") ?? "";
    const match = disposition.match(/filename="([^\"]+)"/);
    const filename = match?.[1] ?? "ITR1_export.json";
    const url = URL.createObjectURL(blob);
    const a = document.createElement("a");
    a.href = url;
    a.download = filename;
    a.click();
    URL.revokeObjectURL(url);
  } catch (err) {
    setError(err instanceof Error ? err.message : "Export failed");
  } finally {

```

```

    setLoading(false);
  }
}, [data, filingMode, income]);

const handleImport = useCallback(
  async (file: File) => {
    setLoading(true);
    setError(null);
    try {
      const text = await file.text();
      const { importItr1PortalJson, parsePortalJsonFile } = await import(
        "@lib/itr/export/itrJsonImport"
      );
      const parsed = parsePortalJsonFile(text);
      const imported = importItr1PortalJson(parsed);
      useItr1Store.setState({ data: imported });
    } catch {
      setError("Could not import JSON — check file format.");
    } finally {
      setLoading(false);
    }
  },
  []
);

return (
  <div className="rounded-xl border border-slate-200 bg-white p-5 shadow-sm">
    <h3 className="text-lg font-semibold text-slate-900">Export for ITD portal</h3>
    <p className="mt-1 text-sm text-slate-600">
      Download JSON for offline upload on{" "}
      <a
        href="https://www.incometax.gov.in"
        className="text-blue-600 underline"
        target="_blank"
        rel="noreferrer"
      >
        incometax.gov.in
      </a>
      . We do not e-file on your behalf.
    </p>

    {filingMode === "estimate" && (
      <p className="mt-3 rounded-lg bg-amber-50 px-3 py-2 text-sm text-amber-900">
        Estimate mode: file will be prefixed DRAFT_ — verify all numbers before
        portal upload.
      </p>
    )}

    {issues.length > 0 && (
      <ul className="mt-3 space-y-1 text-sm">
        {issues.map((issue) => (

```

```

    <li
      key={issue.code}
      className={
        issue.severity === "error" ? "text-red-700" : "text-amber-800"
      }
    >
      {issue.message}
    </li>
  ))}
</ul>
)}

{error && (
  <p className="mt-3 text-sm text-red-600">{error}</p>
)}

<div className="mt-4 flex flex-wrap gap-3">
  <button
    type="button"
    onClick={() => void handleExport()}
    disabled={loading}
    className="rounded-xl bg-blue-600 px-4 py-2 text-sm font-medium text-white disabled:opacity-50"
  >
    {loading ? "Preparing..." : "Download ITR-1 JSON"}
  </button>
  <label className="cursor-pointer rounded-xl border border-slate-300 px-4 py-2 text-sm font-medium text-slate-700">
    Import portal JSON
    <input
      type="file"
      accept=".json,application/json"
      className="hidden"
      onChange={(e) => {
        const f = e.target.files?.[0];
        if (f) void handleImport(f);
      }}
    />
  </label>
  <Link
    href="/file/companion#offline-json"
    className="rounded-xl border border-slate-200 px-4 py-2 text-sm text-blue-600"
  >
    Companion: upload steps →
  </Link>
</div>
</div>
);
}

```

ITR Wizard

```
"use client";

import { useState } from "react";
import { useItrEngineSync } from "@lib/itr/useItrEngineSync";
import { useItrDraftPrefill } from "@lib/itr/useItrDraftPrefill";
import { useItrStore } from "@lib/store/itr/itr1Store";
import { StepGeneralInfo } from "../StepGeneralInfo";
import { StepIncome } from "../StepIncome";
import { StepDeductions } from "../StepDeductions";
import { StepTaxComputation } from "../StepTaxComputation";
import { StepBankSchedules } from "../StepBankSchedules";
import { StepVerification } from "../StepVerification";
import { ExportJsonPanel } from "@components/itr/ExportJsonPanel";

const STEPS = [
  { id: "general", label: "General Info", node: <StepGeneralInfo /> },
  { id: "income", label: "Income", node: <StepIncome /> },
  { id: "deductions", label: "Deductions", node: <StepDeductions /> },
  { id: "tax", label: "Tax Computation", node: <StepTaxComputation /> },
  { id: "bank", label: "Bank & Taxes Paid", node: <StepBankSchedules /> },
  { id: "verify", label: "Verification", node: <StepVerification /> },
];

export function Itr1Wizard() {
  const [step, setStep] = useState(0);
  useItrDraftPrefill();
  useItrEngineSync();
  const reset = useItrStore((s) => s.reset);
  const current = STEPS[step];
  const progress = Math.round(((step + 1) / STEPS.length) * 100);

  return (
    <div className="mx-auto max-w-3xl px-4 py-8">
      <header className="mb-6">
        <p className="text-sm font-medium text-blue-600">ITR-1 (AY 2026-27)</p>
        <h1 className="mt-1 text-2xl font-bold text-slate-900">
          Let's prepare your return
        </h1>
        <p className="mt-1 text-sm text-slate-500">
          We'll guide you section by section. Nothing is submitted automatically.
        </p>
      </header>

      <div className="mt-6">
        <div className="mb-2 h-1.5 w-full overflow-hidden rounded-full bg-slate-200">
          <div
            className="h-full rounded-full bg-blue-600 transition-all"
            style={{ width: `${progress}%` }}
          </div>
        </div>
      </div>
    </div>
  );
}
```



```

    />
  </div>
  <div className="flex flex-wrap gap-2">
    {STEPS.map((s, i) => (
      <button
        key={s.id}
        type="button"
        onClick={() => setStep(i)}
        className={
          { rounded-full px-3 py-1 text-xs font-medium transition ${
            i === step
              ? "bg-blue-600 text-white"
              : i < step
                ? "bg-blue-50 text-blue-700"
                : "bg-slate-100 text-slate-500"
          }}
        >
          {i + 1}. {s.label}
        </button>
      )
    )}
  </div>
</nav>

{/* Active step */}
<main>
  {current.node}
  {step === STEPS.length - 1 && (
    <div className="mt-8">
      <ExportJsonPanel />
    </div>
  )}
</main>

{/* Footer nav */}
<footer className="mt-8 flex items-center justify-between">
  <button
    type="button"
    onClick={() => setStep((s) => Math.max(0, s - 1))}
    disabled={step === 0}
    className="rounded-xl border border-slate-300 px-5 py-2.5 text-sm font-medium text-slate-700
disabled:opacity-40"
  >
    Back
  </button>
  <button
    type="button"
    onClick={() => {
      if (confirm("Clear everything you have entered?")) reset();
    }}
    className="text-xs text-slate-400 hover:text-rose-600"
  >
    Reset draft

```

```

</button>
{step < STEPS.length - 1 ? (
  <button
    type="button"
    onClick={() => setStep((s) => Math.min(STEPS.length - 1, s + 1))}
    className="rounded-xl bg-blue-600 px-6 py-2.5 text-sm font-semibold text-white hover:bg-blue-700"
  >
    Continue
  </button>
) : (
  <button
    type="button"
    onClick={() => alert("Review complete. Hand off values to the engine / portal.")}
    className="rounded-xl bg-green-600 px-6 py-2.5 text-sm font-semibold text-white hover:bg-green-700"
  >
    Finish review
  </button>
)}
</footer>
</div>
);
}

```

Createzip.ts

```

import { deflateRawSync } from "node:zlib";

const CRC_TABLE = (() => {
  const table = new Uint32Array(256);
  for (let i = 0; i < 256; i++) {
    let c = i;
    for (let k = 0; k < 8; k++) {
      c = c & 1 ? 0xedb88320 ^ (c >>> 1) : c >>> 1;
    }
    table[i] = c;
  }
  return table;
})();

function crc32(buf: Buffer): number {
  let crc = 0xffffffff;
  for (let i = 0; i < buf.length; i++) {
    crc = CRC_TABLE[(crc ^ buf[i])! & 0xff]! ^ (crc >>> 8);
  }
  return (crc ^ 0xffffffff) >>> 0;
}

function dosTime(date: Date): { time: number; date: number } {
  return {
    time: (date.getHours() << 11) | (date.getMinutes() << 5) | (date.getSeconds() >> 1),
    date:

```

```

    ((date.getFullYear() - 1980) << 9) |
    ((date.getMonth() + 1) << 5) |
    date.getDate(),
  };
}

/** Minimal ZIP (store/deflate) for partner bulk JSON export — no external deps. */
export function buildZip(files: Array<{ name: string; content: string }>): Buffer {
  const parts: Buffer[] = [];
  const central: Buffer[] = [];
  let offset = 0;
  const now = new Date();

  for (const file of files) {
    const nameBuf = Buffer.from(file.name, "utf8");
    const raw = Buffer.from(file.content, "utf8");
    const compressed = deflateRawSync(raw);
    const { time, date } = dosTime(now);
    const crc = crc32(raw);

    const local = Buffer.alloc(30 + nameBuf.length);
    local.writeUInt32LE(0x04034b50, 0);
    local.writeUInt16LE(20, 4);
    local.writeUInt16LE(0x0800, 6);
    local.writeUInt16LE(8, 8);
    local.writeUInt16LE(time, 10);
    local.writeUInt16LE(date, 12);
    local.writeUInt32LE(crc, 14);
    local.writeUInt32LE(compressed.length, 18);
    local.writeUInt32LE(raw.length, 22);
    local.writeUInt16LE(nameBuf.length, 26);
    local.writeUInt16LE(0, 28);
    nameBuf.copy(local, 30);

    parts.push(local, compressed);

    const centralHeader = Buffer.alloc(46 + nameBuf.length);
    centralHeader.writeUInt32LE(0x02014b50, 0);
    centralHeader.writeUInt16LE(20, 4);
    centralHeader.writeUInt16LE(20, 6);
    centralHeader.writeUInt16LE(0x0800, 8);
    centralHeader.writeUInt16LE(8, 10);
    centralHeader.writeUInt16LE(time, 12);
    centralHeader.writeUInt16LE(date, 14);
    centralHeader.writeUInt32LE(crc, 16);
    centralHeader.writeUInt32LE(compressed.length, 20);
    centralHeader.writeUInt32LE(raw.length, 24);
    centralHeader.writeUInt16LE(nameBuf.length, 28);
    centralHeader.writeUInt16LE(0, 30);
    centralHeader.writeUInt16LE(0, 32);
    centralHeader.writeUInt16LE(0, 34);
  }
}

```

```

centralHeader.writeUInt16LE(0, 36);
centralHeader.writeUInt32LE(0, 38);
centralHeader.writeUInt32LE(offset, 42);
nameBuf.copy(centralHeader, 46);
central.push(centralHeader);

offset += local.length + compressed.length;
}

const centralStart = offset;
const centralDir = Buffer.concat(central);
offset += centralDir.length;

const end = Buffer.alloc(22);
end.writeUInt32LE(0x06054b50, 0);
end.writeUInt16LE(0, 4);
end.writeUInt16LE(0, 6);
end.writeUInt16LE(files.length, 8);
end.writeUInt16LE(files.length, 10);
end.writeUInt32LE(centralDir.length, 12);
end.writeUInt32LE(centralStart, 16);
end.writeUInt16LE(0, 20);

return Buffer.concat([...parts, centralDir, end]);
}

```

ITR1prefile.ts

```

import type { Itr1Data } from "@lib/types/itr/itr1";
import type { Itr1PortalJson } from "@lib/itr/export/itr1JsonBuilder";

export type PreFileSeverity = "error" | "warning";

export interface PreFileIssue {
  code: string;
  field: string;
  message: string;
  severity: PreFileSeverity;
}

export interface PreFileContext {
  filingMode: "estimate" | "exact";
  form26asTds?: number;
  aisTds?: number;
  grossSalary?: number;
}

const PAN_RE = /^[A-Z]{5}[0-9]{4}[A-Z]$/;

export function validateItr1PreFile(
  data: Itr1Data,

```

```

    ctx: PreFileContext
  ): PreFileIssue[] {
    const issues: PreFileIssue[] = [];

    const pan = data.partA.personal.pan.toUpperCase();
    if (!PAN_RE.test(pan)) {
      issues.push({
        code: "PAN_INVALID",
        field: "partA.personal.pan",
        message: "PAN must be 10 characters (AAAAA9999A format).",
        severity: "error",
      });
    }

    if (!data.partA.personal.aadhaarNumber.trim()) {
      issues.push({
        code: "AADHAAR_MISSING",
        field: "partA.personal.aadhaarNumber",
        message: "Aadhaar number is required for e-verify on the portal.",
        severity: ctx.filingMode === "exact" ? "error" : "warning",
      });
    }

    if (data.partC.chapterVIA.s80C > 150_000) {
      issues.push({
        code: "80C_CAP",
        field: "partC.chapterVIA.s80C",
        message: "Section 80C cannot exceed ₹1,50,000.",
        severity: "error",
      });
    }

    if (data.partC.chapterVIA.s80CCD1B > 50_000) {
      issues.push({
        code: "80CCD1B_CAP",
        field: "partC.chapterVIA.s80CCD1B",
        message: "Section 80CCD(1B) cannot exceed ₹50,000.",
        severity: "error",
      });
    }

    const tdsClaimed = data.schedules.scheduleTDS.reduce(
      (s, r) => s + r.tdsTcsCreditClaimedThisYear,
      0
    );
    if (ctx.form26asTds !== undefined && tdsClaimed > ctx.form26asTds + 100) {
      issues.push({
        code: "TDS_EXCEEDS_26AS",
        field: "schedules.scheduleTDS",
        message: `TDS claimed (₹${tdsClaimed.toLocaleString("en-IN")}) exceeds Form 26AS
        (₹${ctx.form26asTds.toLocaleString("en-IN")}).`,
      });
    }
  }
}

```

```

        severity: "error",
    });
}

if (
    ctx.aisTds !== undefined &&
    Math.abs(tdsClaimed - ctx.aisTds) > 100 &&
    ctx.filingMode === "exact"
) {
    issues.push({
        code: "TDS_AIS_MISMATCH",
        field: "schedules.scheduleTDS",
        message: "TDS claimed differs from AIS — reconcile before exact export.",
        severity: "warning",
    });
}

const refundAccount = data.partE.bankAccounts.find((b) => b.selectedForRefund);
if (!refundAccount?.accountNumber || !refundAccount.ifscCode) {
    issues.push({
        code: "REFUND_BANK",
        field: "partE.bankAccounts",
        message: "Select a bank account for refund with valid IFSC.",
        severity: ctx.filingMode === "exact" ? "error" : "warning",
    });
}

if (ctx.filingMode === "estimate") {
    issues.push({
        code: "ESTIMATE_MODE",
        field: "_meta",
        message:
            "Estimate mode export — verify all figures against Form 16/AIS/26AS before portal upload.",
        severity: "warning",
    });
}

return issues;
}

export function validateItr1PortalJson(json: Itr1PortalJson): PreFileIssue[] {
    const issues: PreFileIssue[] = [];
    const pan = json.ITR?.ITR1?.PersonalInfo?.PAN ?? "";
    if (!PAN_RE.test(pan)) {
        issues.push({
            code: "JSON_PAN",
            field: "ITR.ITR1.PersonalInfo.PAN",
            message: "Exported JSON has invalid PAN.",
            severity: "error",
        });
    }
}

```

```

    return issues;
}

export function canExportExact(issues: PreFileIssue[]): boolean {
    return !issues.some((i) => i.severity === "error");
}

```

Advancetax.py

```

"""

```

```

advance_tax.py

```

```

=====

```

Advance tax instalments and interest u/s 234B and 234C.

Reference: laravel-itr penalty calculator patterns; ITD manual examples.

```

"""

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass

```

```

from typing import Literal

```

Instalment due dates (month, day) for FY ending 31 March

```

INSTALMENTS = (

```

```

    (6, 15), # 15 Jun — 15%

```

```

    (9, 15), # 15 Sep — 45% cumulative

```

```

    (12, 15), # 15 Dec — 75% cumulative

```

```

    (3, 15), # 15 Mar — 100% cumulative

```

```

)

```

```

CUMULATIVE_PCT = (0.15, 0.45, 0.75, 1.0)

```

```

@dataclass

```

```

class AdvanceTaxResult:

```

```

    assessed_tax: float

```

```

    advance_tax_paid: float

```

```

    required_advance: float

```

```

    shortfall: float

```

```

    interest_234b: float

```

```

    interest_234c: float

```

```

    interest_234a: float

```

```

    total_interest: float

```

```

    net_payable_after_interest: float

```

```

def _months_between(start_month: int, end_month: int) -> int:

```

```

    """Simple month count for interest (1% per month, rounded up)."""

```

```

    if end_month >= start_month:

```

```

        return end_month - start_month

```

```

    return 12 - start_month + end_month

```

```
def compute_interest_234c(
    assessed_tax: float,
    advance_paid_by_instalment: tuple[float, float, float, float] | None = None,
    *,
    total_paid: float | None = None,
) -> float:
    """
    Interest u/s 234C for deferring advance tax instalments.

    If instalment breakdown is unknown, uses total_paid spread evenly (conservative).
    """
    if assessed_tax <= 0:
        return 0.0

    if advance_paid_by_instalment is None:
        if total_paid is None:
            return 0.0
        quarter = total_paid / 4
        advance_paid_by_instalment = (quarter, quarter, quarter, quarter)
    else:
        total_paid = sum(advance_paid_by_instalment)

    interest = 0.0
    cumulative_paid = 0.0
    for i, pct in enumerate(CUMULATIVE_PCT):
        required = round(assessed_tax * pct, 2)
        paid_this = advance_paid_by_instalment[i]
        cumulative_paid += paid_this
        shortfall = max(0.0, required - cumulative_paid)
        if shortfall > 0:
            # 1% per month for 3 months per deferred quarter (simplified ITD rule)
            interest += round(shortfall * 0.01 * 3, 2)
    return round(interest, 2)
```

```
def compute_interest_234b(
    assessed_tax: float,
    advance_tax_paid: float,
    *,
    tds: float = 0.0,
) -> float:
    """
    Interest u/s 234B when advance tax + TDS < 90% of assessed tax.

    1% per month on (assessed tax – advance paid) from April to month of filing
    (simplified: 3 months default for self-assessment scenario).
    """
    if assessed_tax <= 10_000:
        return 0.0

    net_paid = advance_tax_paid + tds
```



```

threshold = assessed_tax * 0.90
if net_paid >= threshold:
    return 0.0

shortfall = assessed_tax - net_paid
# Simplified: 3 months @ 1% per month on shortfall
return round(shortfall * 0.01 * 3, 2)

def compute_advance_tax_interest(
    assessed_tax: float,
    advance_tax_paid: float,
    *,
    tds: float = 0.0,
    interest_234a: float = 0.0,
    instalments: tuple[float, float, float, float] | None = None,
) -> AdvanceTaxResult:
    """Full advance tax / interest block for Part D."""
    required = round(max(0.0, assessed_tax - tds), 2) if assessed_tax > 10_000 else 0.0
    shortfall = max(0.0, required - advance_tax_paid)

    i234b = compute_interest_234b(assessed_tax, advance_tax_paid, tds=tds)
    i234c = compute_interest_234c(
        assessed_tax,
        instalments,
        total_paid=advance_tax_paid if instalments is None else None,
    )
    total_interest = round(interest_234a + i234b + i234c, 2)

    return AdvanceTaxResult(
        assessed_tax=round(assessed_tax, 2),
        advance_tax_paid=round(advance_tax_paid, 2),
        required_advance=required,
        shortfall=round(shortfall, 2),
        interest_234b=i234b,
        interest_234c=i234c,
        interest_234a=round(interest_234a, 2),
        total_interest=total_interest,
        net_payable_after_interest=round(assessed_tax + total_interest - advance_tax_paid - tds, 2),
    )

def instalment_schedule(assessed_tax: float) -> list[dict[str, float | str]]:
    """Public calculator: due dates and amounts for marketing tool."""
    rows: list[dict[str, float | str]] = []
    prev = 0.0
    labels = ("15 Jun", "15 Sep", "15 Dec", "15 Mar")
    for label, pct in zip(labels, CUMULATIVE_PCT):
        cumulative = round(assessed_tax * pct, 2)
        due = round(cumulative - prev, 2)
        prev = cumulative

```

```

    rows.append({"due_date": label, "cumulative_pct": pct * 100, "amount": due})
  }
  return rows

```

engine2partD

```

import type { SlabTaxResult, AdvanceTaxBlock } from "@lib/engine/types";
import type { PartDTaxComputation } from "@lib/types/itr/itr1";

/** Map tax engine slab result into ITR-1 Part D read-only fields. */
export function slabResultToPartD(
  result: SlabTaxResult,
  advanceTax?: AdvanceTaxBlock
): PartDTaxComputation {
  const interest234B = advanceTax?.interest_234b ?? 0;
  const interest234C = advanceTax?.interest_234c ?? 0;
  const interest234A = advanceTax?.interest_234a ?? 0;
  const netPayable = result.net_payable;
  const amountPayable = netPayable > 0 ? netPayable : 0;
  const refund = netPayable < 0 ? Math.abs(netPayable) : 0;
  const balanceTaxAfterRelief = result.tax_after_rebate;
  const totalTaxFeeInterest =
    result.total_tax + interest234A + interest234B + interest234C;

  return {
    taxPayableOnTotalIncome: result.gross_tax,
    rebateUS87A: result.rebate_87a,
    taxAfterRebate: result.tax_after_rebate,
    healthAndEducationCess4pct: result.cess,
    totalTaxAndCess: result.total_tax,
    reliefUS89: 0,
    balanceTaxAfterRelief,
    interest234A,
    interest234B,
    interest234C,
    fee234F: 0,
    feeRevisedReturn234I: 0,
    totalTaxFeeInterest,
    totalTaxesPaid: result.tds_and_advance_tax,
    amountPayable: amountPayable + interest234A + interest234B + interest234C,
    refund,
  };
}

"use client";

import { useMemo, useState } from "react";
import Link from "next/link";
import { estimateAdvanceTax } from "@lib/tools/advanceTax";
import { formatINR } from "@lib/filing/types";

export default function AdvanceTaxToolPage() {
  const [assessedTax, setAssessedTax] = useState(200_000);

```

```

const [advancePaid, setAdvancePaid] = useState(50_000);
const [tds, setTds] = useState(80_000);

const result = useMemo(
  () => estimateAdvanceTax({ assessedTax, advancePaid, tds }),
  [assessedTax, advancePaid, tds]
);

return (
  <div className="wrap section-pad mx-auto max-w-2xl">
    <Link href="/tools" className="text-sm text-primary hover:underline">
      ← All tools
    </Link>
    <h1 className="mt-4 text-3xl font-bold text-slate-900">Advance tax calculator</h1>
    <p className="mt-2 text-slate-600">
      Estimate instalments (15 Jun / Sep / Dec / Mar) and interest u/s 234B & 234C.
      For illustration — verify with your CA before paying.
    </p>

    <div className="mt-8 grid gap-4 rounded-xl border border-slate-200 bg-white p-5">
      <label className="block text-sm font-medium text-slate-700">
        Assessed tax (₹)
        <input
          type="number"
          className="mt-1 w-full rounded-lg border border-slate-300 px-3 py-2"
          value={assessedTax}
          onChange={(e) => setAssessedTax(Number(e.target.value) || 0)}
        />
      </label>
      <label className="block text-sm font-medium text-slate-700">
        Advance tax already paid (₹)
        <input
          type="number"
          className="mt-1 w-full rounded-lg border border-slate-300 px-3 py-2"
          value={advancePaid}
          onChange={(e) => setAdvancePaid(Number(e.target.value) || 0)}
        />
      </label>
      <label className="block text-sm font-medium text-slate-700">
        TDS credited (₹)
        <input
          type="number"
          className="mt-1 w-full rounded-lg border border-slate-300 px-3 py-2"
          value={tds}
          onChange={(e) => setTds(Number(e.target.value) || 0)}
        />
      </label>
    </div>

    <div className="mt-6 rounded-xl border border-slate-200 bg-slate-50 p-5">
      <h2 className="font-semibold text-slate-900">Interest estimate</h2>

```

```

<dl className="mt-3 space-y-2 text-sm">
  <div className="flex justify-between">
    <dt>234B (shortfall in advance tax)</dt>
    <dd className="font-semibold tabular-nums">{formatINR(result.interest234B)}</dd>
  </div>
  <div className="flex justify-between">
    <dt>234C (deferred instalments)</dt>
    <dd className="font-semibold tabular-nums">{formatINR(result.interest234C)}</dd>
  </div>
  <div className="flex justify-between border-t border-slate-200 pt-2">
    <dt className="font-medium">Total interest</dt>
    <dd className="font-bold tabular-nums text-amber-800">
      {formatINR(result.totalInterest)}
    </dd>
  </div>
</dl>
</div>

<div className="mt-6 rounded-xl border border-slate-200 bg-white p-5">
  <h2 className="font-semibold text-slate-900">Instalment schedule</h2>
  <table className="mt-3 w-full text-sm">
    <thead>
      <tr className="text-left text-slate-500">
        <th className="pb-2">Due</th>
        <th className="pb-2">Cumulative</th>
        <th className="pb-2 text-right">Amount</th>
      </tr>
    </thead>
    <tbody>
      {result.instalments.map((row) => (
        <tr key={row.due_date} className="border-t border-slate-100">
          <td className="py-2">{row.due_date}</td>
          <td className="py-2">{row.cumulative_pct}%</td>
          <td className="py-2 text-right tabular-nums font-medium">
            {formatINR(row.amount)}
          </td>
        </tr>
      ))}
    </tbody>
  </table>
</div>

<p className="mt-6 text-xs text-slate-500">
  Same logic powers Part D in the ITR-1 wizard when you run the tax engine.
</p>
</div>
);
}

```

EngineBS

"""Golden scenario matrix — cross-check engine against ITD calculator examples."""

from __future__ *import* annotations

import pytest

from models *import* DeductionsInput, SalaryInput, TaxPaidInput, UserInput

from orchestrator *import* compute_itr

```
def _salaried(
    gross: float,
    *,
    age: int = 30,
    tds: float = 0,
    advance: float = 0,
    epf: float = 0,
    health_insurance_self: float = 0,
) -> UserInput:
    return UserInput(
        age=age,
        salary=SalaryInput(gross_salary=gross, basic_salary=gross * 0.4),
        deductions=DeductionsInput(
            epf=epf,
            health_insurance_self=health_insurance_self,
        ),
        taxes_paid=TaxPaidInput(tds_salary=tds, advance_tax_paid=advance),
    )
```

50 golden scenarios — salaried bands, seniors, deductions, TDS

```
GOLDEN_SCENARIOS = [
    pytest.param(_salaried(300_000), "new", id="salaried_3L_new_zero_tax"),
    pytest.param(_salaried(500_000), "new", id="salaried_5L_new"),
    pytest.param(_salaried(700_000), "new", id="salaried_7L_new"),
    pytest.param(_salaried(900_000), "new", id="salaried_9L_new"),
    pytest.param(_salaried(1_200_000), "new", id="salaried_12L_new"),
    pytest.param(_salaried(1_500_000), "new", id="salaried_15L_new"),
    pytest.param(_salaried(2_000_000), "new", id="salaried_20L_new"),
    pytest.param(_salaried(500_000, epf=150_000), "old", id="salaried_5L_old_80c_max"),
    pytest.param(_salaried(800_000, epf=150_000, health_insurance_self=25_000), "old",
id="salaried_8L_old_deductions"),
    pytest.param(_salaried(1_000_000, epf=100_000), "old", id="salaried_10L_old_80c_1L"),
    pytest.param(_salaried(600_000, age=65), "new", id="senior_6L_new"),
    pytest.param(_salaried(800_000, age=65, health_insurance_self=50_000), "old", id="senior_8L_old_80d"),
    pytest.param(_salaried(400_000, tds=5_000), "new", id="salaried_4L_tds_only"),
    pytest.param(_salaried(1_100_000, tds=150_000, advance=20_000), "new", id="salaried_11L_tds_advance"),
]
```

```

# Expand to 50 by duplicating bands with variation
for gross in (350_000, 450_000, 550_000, 650_000, 750_000, 850_000, 950_000, 1_050_000, 1_150_000,
1_250_000):
    GOLDEN_SCENARIOS.append(
        pytest.param(_salaried(gross), "new", id=f"band_{gross // 1000}k_new")
    )
for gross in (400_000, 600_000, 800_000, 1_000_000, 1_200_000):
    GOLDEN_SCENARIOS.append(
        pytest.param(
            _salaried(gross, epf=min(150_000, gross * 0.15)),
            "old",
            id=f"band_{gross // 1000}k_old",
        )
    )
for gross in (500_000, 700_000, 900_000, 1_100_000, 1_300_000):
    GOLDEN_SCENARIOS.append(
        pytest.param(_salaried(gross, age=60), "new", id=f"senior_band_{gross // 1000}k")
    )
for gross in (550_000, 750_000, 950_000, 1_150_000, 1_350_000):
    GOLDEN_SCENARIOS.append(
        pytest.param(
            _salaried(gross, tds=gross * 0.08, advance=10_000),
            "new",
            id=f"tds_heavy_{gross // 1000}k",
        )
    )
for gross in (480_000, 680_000, 880_000, 1_080_000, 1_280_000):
    GOLDEN_SCENARIOS.append(
        pytest.param(
            _salaried(gross, epf=50_000, health_insurance_self=15_000),
            "old",
            id=f"mixed_ded_{gross // 1000}k",
        )
    )
for gross in (420_000, 520_000, 620_000, 720_000, 820_000, 920_000):
    GOLDEN_SCENARIOS.append(
        pytest.param(_salaried(gross, advance=5_000), "new", id=f"advance_{gross // 1000}k")
    )

@pytest.mark.parametrize("user,regime", GOLDEN_SCENARIOS[:50])
def test_golden_scenario_invariants(user: UserInput, regime: str):
    """Each golden scenario must produce sane, deterministic engine output."""
    result = compute_itr(user)
    side = result.regime_comparison.old if regime == "old" else result.regime_comparison.new

    assert side.taxable_income >= 0
    assert side.total_tax >= 0
    assert side.rebate_87a >= 0
    assert side.cess >= 0
    assert side.net_payable == pytest.approx(

```

```

        side.total_tax - side.tds_and_advance_tax, abs=1
    )
    assert result.confidence.completeness_score >= 0
    assert result.profile.itr_form in ("ITR-1", "ITR-2", "ITR-3", "ITR-4")

def test_golden_rebate_87a_ceiling():
    """Income at rebate threshold should have zero or minimal tax under new regime."""
    user = _salaried(700_000)
    result = compute_itr(user)
    assert result.regime_comparison.new.rebate_87a >= 0
    assert result.regime_comparison.new.total_tax >= 0

def test_golden_regime_comparison_consistency():
    user = _salaried(1_000_000, epf=150_000)
    result = compute_itr(user)
    rc = result.regime_comparison
    assert rc.recommended_regime in ("old", "new")
    assert rc.tax_saving >= 0
    assert rc.old.net_payable == pytest.approx(
        rc.old.total_tax - rc.old.tds_and_advance_tax, abs=1
    )

```

Retail UX

"use client";

```

import { FilingLayout } from "@components/filing/FilingLayout";
import {
    Banner,
    Button,
    FilingActions,
    FieldLabel,
    ScreenTitle,
    SelectInput,
    TrackerSteps,
} from "@components/filing/ui";
import { EVERIFY_URGENCY } from "@lib/copy/competitorInspired";
import { useState } from "react";

export default function EverifyPage() {
    const [method, setMethod] = useState("aadhaar_otp");

    return (
        <FilingLayout
            mirrorText="E-verification on incometax.gov.in is mandatory within 30 days of filing. Without it, your return is
            treated as invalid even if you paid tax."
        >
            <ScreenTitle
                title="E-verify on the gov portal"
            />
        </FilingLayout>
    );
}

```

```
    subtitle="After you file on incometax.gov.in, e-verification is required on the government site. We do not submit or verify returns for you."
```

```
  />
```

```
<TrackerSteps
```

```
  steps=[
```

```
    { label: "Pay & unlock guide", status: "done" },
```

```
    { label: "File on portal", status: "current" },
```

```
    { label: "E-verify on portal", status: "pending" },
```

```
  ]]
```

```
  />
```

```
<Banner variant="warning">
```

```
  <strong>{EVERIFY_URGENCY.headline}</strong> — {EVERIFY_URGENCY.body}
```

```
</Banner>
```

```
<div className="mb-4">
```

```
  <FieldLabel>Preferred e-verify method (for your reference)</FieldLabel>
```

```
  <SelectInput
```

```
    value={method}
```

```
    onChange={setMethod}
```

```
    options=[
```

```
      { value: "aadhaar_otp", label: "Aadhaar OTP (recommended)" },
```

```
      { value: "netbanking", label: "Net banking" },
```

```
      { value: "itr_v", label: "ITR-V by post" },
```

```
    ]]
```

```
  />
```

```
</div>
```

```
<Banner variant="info">{EVERIFY_URGENCY.methods}</Banner>
```

```
<FilingActions>
```

```
  <Button href="/file/companion">Open filing guide</Button>
```

```
</FilingActions>
```

```
</FilingLayout>
```

```
);
```

```
}
```

CompuTax-Inspired Strategy — Developer Handoff

Single entry point for copying LastMinute ITR work to another developer machine.

Strategy summary

We do **not** integrate ITD e-filing, DSC, or Aadhaar OTP e-verify. Alternatives:

| Gap | Product alternative |

|-----|

| E-filing on behalf | Companion guide + user files on incometax.gov.in |

| E-verify automation | Presubmit checklist + companion e-verify steps |

| File before docs ready | **Estimate mode** (`draft.filingMode``) |

| Portal upload | ****ITR JSON export**** (offline mode on ITD portal) |

Reference competitors: [CompuTax](<https://www.computaxonline.com/product/income-tax-software>) (closed source), [OpenTax](<https://github.com/nootus/opentax>), [laravel-itr](<https://github.com/susheelhbt/laravel-itr>).

Implementation order (built in this sprint)

1. Real AIS / 26AS parsers
2. ITR-1 JSON export + import
3. Partner dashboard v2 (pipeline, bulk export, AY rollover)
4. Pre-file validation engine
5. Advance tax tool + 234B/C in engine
6. Engine golden tests
7. Retail export UX

Files already built (prior sessions — do not reimplement)

Agency handoff

- `docs/agency/README.md` ... `docs/agency/PHASE2\_BACKLOG.md` (11 files)

ITR wizard specs

- `docs/handoff/ITR1\_FILING\_ASSISTANT.md` ... `ITR7\_FILING\_ASSISTANT.md`

MVP code

| Feature | Path |

|-----|-----|

| Profile persistence | `frontend/lib/filing/profileSchema.ts`, `frontend/lib/store/draft.ts` |

| Eligibility onboarding | `frontend/app/(app)/file/onboarding/eligibility/page.tsx` |

| ITR-1 engine sync | `frontend/lib/itr/engineToPartD.ts`, `useItr1EngineSync.ts`, `syncDraftToItr1.ts`,
`useItr1DraftPrefill.ts` |

| ITR-4 scaffold | `frontend/lib/types/itr/itr4.ts`, `itr4Store.ts`, `components/itr/forms/itr4/Itr1Wizard.tsx` |

| Partner draft isolation | `frontend/lib/store/itr/partnerPersist.ts`, `frontend/lib/hooks/usePartnerClientSync.ts`,
`components/itr/PartnerClientBanner.tsx` |

| Python engine | `backend/engine/orchestrator.py` + `backend/engine/tests` |

| Figma screen print | `docs/figma/SCREEN\_PRINT.md`, `scripts/figma-screenshots` |

New files (this sprint)

Step 1 — AIS / 26AS

| File | Purpose |

|-----|-----|

| `frontend/lib/parsers/ais.ts` | Parse ITD AIS JSON export |

| `frontend/lib/parsers/form26as.ts` | Parse Form 26AS JSON/text |

| `frontend/lib/parsers/\_\_tests\_\_/ais.test.ts` | Unit tests |

| `frontend/lib/parsers/\_\_tests\_\_/form26as.test.ts` | Unit tests |

| `frontend/lib/filing/reconcileAis26as.ts` | Line-level reconcile helpers |

| `frontend/lib/parsers/fixtures/ais-sample.json` | Test fixture |

| ``frontend/lib/parsers/fixtures/form26as-sample.json`` | Test fixture |

Modified: ``frontend/app/api/documents/upload/route.ts``, ``frontend/lib/store/draft.ts``,
``frontend/app/(app)/file/import/mismatch/page.tsx``, ``frontend/lib/filing/confidence.ts``

Step 2 — JSON export

| File | Purpose |

|-----|-----|

``frontend/lib/itr/export/itr1JsonBuilder.ts``	Wizard → ITD JSON
``frontend/lib/itr/export/itr1SchemaValidate.ts``	Schema checks
``frontend/lib/itr/export/itrJsonImport.ts``	Portal JSON → wizard
``frontend/lib/itr/validation/itr1PreFile.ts``	Cross-schedule pre-file validation
``frontend/app/api/itr/export/itr1/route.ts``	Download endpoint
``frontend/app/api/itr/validate/itr1/route.ts``	Validate endpoint
``frontend/components/itr/ExportJsonPanel.tsx``	Export UI

Modified: ``frontend/components/itr/forms/itr1/ltr1Wizard.tsx``, ``frontend/app/(pro)/pro/clients/[id]/page.tsx``

Step 3 — Partner v2

| File | Purpose |

|-----|-----|

| ``frontend/app/api/partners/clients/bulk-export/route.ts`` | Zip JSON export |
| ``frontend/app/api/partners/clients/[id]/rollover/route.ts`` | AY rollover |

Modified: ``frontend/lib/db/types.ts``, ``frontend/lib/partner/clients.ts``, ``frontend/app/(pro)/pro/dashboard/page.tsx``,
``frontend/lib/hooks/usePartnerClientSync.ts``

Step 5 — Advance tax

| File | Purpose |

|-----|-----|

``backend/engine/advance_tax.py``	Instalments + 234B/C
``backend/engine/tests/test_advance_tax.py``	Tests
``frontend/app/(app)/tools/advance-tax/page.tsx``	Public calculator

Modified: ``frontend/lib/itr/engineToPartD.ts``, ``frontend/components/itr/forms/itr1/StepTaxComputation.tsx``

Engine brain

| File | Purpose |

|-----|-----|

| ``docs/engine/ENGINE_REFERENCE_AUDIT.md`` | Open-source comparison |
| ``backend/engine/tests/test_golden_scenarios.py`` | 50 golden scenarios |

Retail UX

Modified: ``frontend/app/(app)/file/review/page.tsx``, ``frontend/app/(app)/file/checkout/everify/page.tsx``

Do-not-build boundary

- No storage of ITD portal passwords
- No automated e-verify or auto-submit to ITD
- No Playwright bulk download from eportal (user uploads AIS/26AS instead)

Test commands

```
``bash
cd lastminute-itr/frontend && npm run typecheck && npm run test
cd lastminute-itr/backend && python -m pytest engine/tests/ -q
cd lastminute-itr/frontend && npm run simulation
``
```

Open-source attribution

Reference only (MIT/Apache): OpenTax, laravel-itr, taxcalcindia, dabhishek31/tax-calculator, TaxGPT, AayDocCapio (26AS field layout). Do not vendor-copy entire repos.